

DERLEYİCİLER (COMPILERS)

İçindekiler

- [1. Giriş](#)
- [2. Derleyici Nedir?](#)
- [3. Derleyicilerin Tarihçesi](#)
- [4. Derleyici Aşamaları](#)
 - [o Sözcüksel Analiz \(Lexical Analysis\)](#)
 - [o Sözdizimsel Analiz \(Syntax Analysis\)](#)
 - [o Semantik Analiz \(Semantic Analysis\)](#)
 - [o Ara Kod Üretimi \(Intermediate Code Generation\)](#)
 - [o Kod Optimizasyonu \(Code Optimization\)](#)
 - [o Hedef Kod Üretimi \(Code Generation\)](#)
- [5. Derleyici Türleri](#)
- [6. Yaygın Kullanılan Derleyiciler](#)
- [7. Derleyicilerin Geleceği](#)
- [8. Kaynakça](#)

****SAYFA 1****

Giriş

Bilgisayar bilimlerinin en temel ve önemli alanlarından biri olan derleyiciler, programlama dillerinde yazılan kodları bilgisayarın anlayabileceği makine diline çeviren yazılımlardır. Bu doküman, derleyicilerin ne olduğunu, nasıl çalıştığını, tarihsel gelişimini ve günümüzdeki kullanım alanlarını detaylı bir şekilde incelemektedir.

Derleyici Nedir?

Derleyici, yüksek seviyeli bir programlama dilinde yazılmış kaynak kodunu, bilgisayarın işleyebileceği makine diline veya daha düşük seviyeli bir ara dile dönüştüren bir yazılımdır. Temelde bir çevirmendir ve kullanıcı tarafından yazılan kodu bilgisayarın anlayabileceği bir formata çevirir.

Derleyiciler, programlama dilinin kurallarına göre yazılan kodun doğruluğunu kontrol eder, hataları tespit eder ve programcıya bildirir. Ayrıca kod optimizasyonu yaparak, programın daha hızlı ve daha az kaynak kullanarak çalışmasını sağlar.

Derleyicinin temel işlevi, insan tarafından anlaşılabilir kodun, bilgisayar tarafından anlaşılabilir bir dile dönüştürülmesidir. Bu işlevi nedeniyle derleyiciler, programlama dilleri ve bilgisayar mimarisi arasında bir köprü vazifesi görür.

****SAYFA 2****

Derleyicilerin Tarihçesi

İlk derleyici, 1952 yılında Grace Hopper tarafından geliştirilen A-0 Sistemidir. Bu, modern anlamda bir derleyiciden çok bir yükleyici veya bağlayıcı olarak nitelendirilebilir. İlk tam işlevsel derleyici ise 1957 yılında, FORTRAN programlama dili için John Backus ve ekibi tarafından IBM için geliştirilmiştir.

1960'larda, derleyici tasarımının teorisi gelişmeye başladı ve ALGOL gibi diller için derleyiciler ortaya çıktı. Bu dönemde, derleyici teknolojisi hızla ilerledi ve daha karmaşık diller için derleyiciler geliştirildi.

1970'ler ve 1980'ler, C, Pascal ve Ada gibi dillerin popülerleşmesiyle birlikte derleyici teknolojisinin olgunlaşma dönemi oldu. Bu dönemde, derleyici optimizasyonu ve hata tespiti gibi konularda önemli ilerlemeler kaydedildi.

1990'lardan itibaren, nesne yönelimli programlama dillerinin yaygınlaşmasıyla birlikte, C++, Java ve C# gibi diller için gelişmiş derleyiciler ortaya çıktı. Ayrıca, Just-In-Time (JIT) derleme gibi yeni derleyici teknolojileri geliştirildi.

Günümüzde, LLVM ve GCC gibi açık kaynaklı derleyici altyapıları, modern derleyici teknolojisinin temelini oluşturmaktadır. Bu altyapılar, çok sayıda programlama dili için derleyici geliştirmeyi kolaylaştırmış ve derleyici optimizasyonu alanında önemli ilerlemeler sağlamıştır.

Derleyici Aşamaları

Modern bir derleyici, kaynak kodunu makine koduna dönüştürürken genellikle aşağıdaki aşamalardan geçer:

****SAYFA 3******Sözcüksel Analiz**

Sözcüksel analiz, kaynak kodunun karakterlerden oluşan dizisini tokenlar (jetonlar) olarak adlandırılan anlamlı birimlere dönüştürme işlemidir. Bu aşamada, derleyici kaynak kodu tarar ve dildeki anahtar kelimeler, tanımlayıcılar, operatörler ve diğer sözdizimsel öğeleri tanımlar.

Örneğin, $x = y + 5;$ ifadesi sözcüksel analizde şu tokenlara ayrılabilir:

- x (tanımlayıcı)
- $=$ (atama operatörü)
- y (tanımlayıcı)
- $+$ (toplama operatörü)
- 5 (sayısal sabit)
- $;$ (noktalı virgül)

Sözcüksel analiz, genellikle tarayıcı (scanner) veya leksör (lexer) olarak adlandırılan bir bileşen tarafından gerçekleştirilir.

Sözdizimsel Analiz

Sözcüksel analizden elde edilen tokenlar, sözdizimsel analiz aşamasında programlama dilinin gramer kurallarına göre analiz edilir. Bu aşamada, tokenlar arasındaki ilişkiler belirlenir ve bir sözdizimi ağacı (parse tree) veya soyut sözdizimi ağacı (abstract syntax tree, AST) oluşturulur.

Sözdizimsel analiz, programın yapısını anlamak ve dilbilgisi hatalarını tespit etmek için önemlidir. Bu aşamada, örneğin parantezlerin eşleşip eşleşmediği, ifadelerin doğru bir şekilde sonlandırılıp sonlandırılmadığı gibi kontroller yapılır.

****SAYFA 4****

Semantik Analiz

Semantik analiz aşamasında, programın anlamsal doğruluğu kontrol edilir. Bu aşamada, tür uyumluluğu, değişken tanımları, kapsamlar ve diğer semantik özellikler incelenir.

Örneğin, bir değişkenin tanımlanmadan kullanılması, bir fonksiyonun yanlış sayıda veya türde argüman ile çağırılması gibi durumlar semantik analiz aşamasında tespit edilir.

Semantik analiz sonucunda, genellikle derleyicinin sonraki aşamalarında kullanılacak olan zenginleştirilmiş bir soyut sözdizimi ağacı ve sembol tablosu oluşturulur.

Ara Kod Üretimi

Semantik analizden sonra, derleyici genellikle bir ara kod (intermediate code) üretir. Bu ara kod, kaynak kodundan daha düşük seviyeli ancak hedef makineden bağımsız bir temsil biçimidir.

Ara kod, optimizasyon işlemleri için daha uygun bir format sağlar ve farklı hedef mimariler için kod üretimini kolaylaştırır. Yaygın ara kod biçimleri arasında üç adresli kod, bytecode ve LLVM IR (Intermediate Representation) bulunur.

Kod Optimizasyonu

Kod optimizasyonu aşamasında, ara kod analiz edilir ve daha verimli bir hale getirilir. Bu aşamanın amacı, programın performansını artırmak, bellek kullanımını azaltmak ve genel olarak daha verimli bir kod elde etmektir.

Optimizasyon teknikleri arasında sabit ifade değerlendirmesi, ölü kod eliminasyonu, döngü optimizasyonu, register ataması ve birçok diğer teknik bulunur. Modern derleyiciler, çok sayıda ve karmaşık optimizasyon tekniği uygulayabilir.

****SAYFA 5****

Hedef Kod Üretimi

Derleyicinin son aşaması, optimize edilmiş ara kodun hedef mimarinin makine koduna dönüştürülmesidir. Bu aşamada, derleyici hedef mimarinin özelliklerini, komut setini ve register yapısını dikkate alarak makine kodu üretir.

Hedef kod, doğrudan çalıştırılabilir bir program (executable) olabileceği gibi, bir nesne dosyası (object file) da olabilir. Nesne dosyaları daha sonra bir bağlayıcı (linker) tarafından birleştirilerek çalıştırılabilir program oluşturulur.

Derleyici Türleri

Derleyiciler, çeşitli kriterlere göre sınıflandırılabilir:

1. Kaynak ve Hedef Dil Bazında:

- **Çapraz Derleyici (Cross Compiler):** Farklı bir işletim sistemi veya mimari için kod üreten derleyicilerdir.
- **Kaynak-Kaynak Derleyici (Source-to-Source Compiler):** Bir programlama dilinden başka bir programlama diline çeviri yapan derleyicilerdir.
- **Just-In-Time (JIT) Derleyici:** Program çalışırken, ara kodu gerçek zamanlı olarak makine koduna çeviren derleyicilerdir.

2. Optimizasyon Seviyesine Göre:

- **Debug Derleyiciler:** Hata ayıklama için optimize edilmemiş kod üreten derleyicilerdir.
- **Optimizasyon Derleyicileri:** Farklı seviyelerde optimizasyon uygulayan derleyicilerdir.

3. Derleme Stratejisine Göre:

- **Tek Geçişli Derleyiciler (Single-Pass Compilers):** Kaynak kodu tek bir geçişte işleyen derleyicilerdir.
 - **Çok Geçişli Derleyiciler (Multi-Pass Compilers):** Kaynak kodu birden fazla geçişte işleyen derleyicilerdir.
-

****SAYFA 6****

Yaygın Kullanılan Derleyiciler

Günümüzde yaygın olarak kullanılan bazı derleyiciler şunlardır:

1. GCC (GNU Compiler Collection)

C, C++, Objective-C, Fortran, Ada ve diğer diller için açık kaynaklı bir derleyici koleksiyonudur. 1987 yılında Richard Stallman tarafından başlatılan GCC projesi, günümüzde en yaygın kullanılan açık kaynaklı derleyicilerden biridir.

2. Clang

LLVM projesinin bir parçası olan, C, C++, Objective-C ve Objective-C++ dilleri için modern bir derleyicidir. Apple tarafından desteklenen Clang, modüler bir yapıya sahiptir ve GCC'ye göre daha hızlı derleme süreleri ve daha anlaşılır hata mesajları sağlamaktadır.

3. MSVC (Microsoft Visual C++)

Microsoft tarafından geliştirilen, C ve C++ dilleri için bir derleyicidir. Windows ortamında yazılım geliştirmek için özel olarak tasarlanmıştır ve Visual Studio IDE ile entegre çalışır.

4. Java Compiler (javac)

Java programlama dili için standart derleyicidir, Java bytecode üretir. Bu bytecode, Java Virtual Machine (JVM) üzerinde platform bağımsız olarak çalıştırılabilir.

5. Rust Compiler (rustc)

Rust programlama dili için resmi derleyicidir. LLVM altyapısını kullanan Rust derleyicisi, bellek güvenliği ve eşzamanlılık özelliklerine sahip kod üretmek için tasarlanmıştır.

6. GHC (Glasgow Haskell Compiler)

Haskell programlama dili için en yaygın kullanılan derleyicidir. Fonksiyonel programlama paradigmasına uygun olarak tasarlanmış güçlü bir tür sistemi ve yüksek seviyeli optimizasyon özellikleri sunar.

7. Go Compiler (gc)

Go programlama dili için resmi derleyicidir. Google tarafından geliştirilen Go derleyicisi, hızlı derleme süreleri ve eşzamanlı programlama için optimize edilmiş kod üretimi ile bilinir.

****SAYFA 7****

Derleyicilerin Geleceđi

Derleyici teknolojisi sürekli olarak gelişmektedir ve gelecekte aşağıdaki alanlarda önemli ilerlemeler beklenmektedir:

1. Daha Akıllı Optimizasyonlar

Makine öğrenimi ve yapay zeka teknikleri kullanılarak daha gelişmiş ve bağlama duyarlı optimizasyonlar yapılabilecektir.

2. Paralel ve Dağıtık Derleme

Çok çekirdekli işlemciler ve dağıtık sistemlerin yaygınlaşmasıyla, derleme sürecinin paralelleştirilmesi ve hızlandırılması mümkün olacaktır.

3. Daha Hızlı Derleme Süreleri

Modern yazılım projelerinin büyüklüğü nedeniyle, derleme sürelerini azaltmak için yeni teknikler geliştirilecektir.

4. Dil Agnostik Derleyici Altyapıları

LLVM gibi, birden fazla programlama dili için kullanılabilen derleyici altyapıları daha da yaygınlaşacaktır.

5. Güvenlik Odaklı Derleyiciler

Güvenlik açıklarını tespit eden ve önleyen, güvenlik odaklı derleyici özellikleri daha da gelişecektir.

6. Heterojenik Sistemler için Derleyiciler

GPU, FPGA ve diğer özel donanımlar için kod üretebilen derleyiciler daha yaygın hale gelecektir.

Kaynakça

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd Edition). Addison Wesley.
2. Cooper, K. D., & Torczon, L. (2011). Engineering a Compiler (2nd Edition). Morgan Kaufmann.
3. Muchnick, S. S. (1997). Advanced Compiler Design and Implementation. Morgan Kaufmann.
4. Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J., & Langendoen, K. (2012). Modern Compiler Design (2nd Edition). Springer.
5. Appel, A. W. (2004). Modern Compiler Implementation in Java (2nd Edition). Cambridge University Press.
6. "The LLVM Compiler Infrastructure." LLVM.org. <https://llvm.org/>
7. "GCC, the GNU Compiler Collection." GNU.org. <https://gcc.gnu.org/>
8. Scott, M. L. (2015). Programming Language Pragmatics (4th Edition). Morgan Kaufmann.